

Fundamentals of Data Science

Lecture 3 - Data retrieval

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2026

¹Co-founder @ Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

Table of contents

1. Get motivated for (biggish) data
2. How to speed things up
 - Only use what you need
 - Choose the right data types
 - Optimise your queries
 - In-memory vs. on disk
 - (Relational) Database management systems & SQL
3. Python and DBs
4. Data storage & Mngt. landscape
5. Adding to our repo structure: .gitignore, pre-commit, type hints

Get motivated for (biggish) data

Remember: DSs spend most their time on data

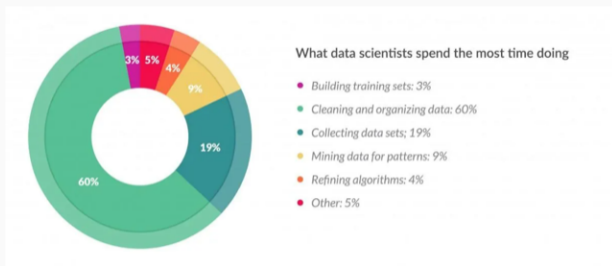


Figure 1: [Forbes article \(2016\)](#)

Remember: DSs spend most their time on data

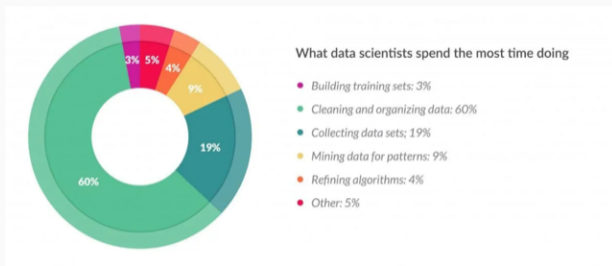


Figure 1: [Forbes article \(2016\)](#)

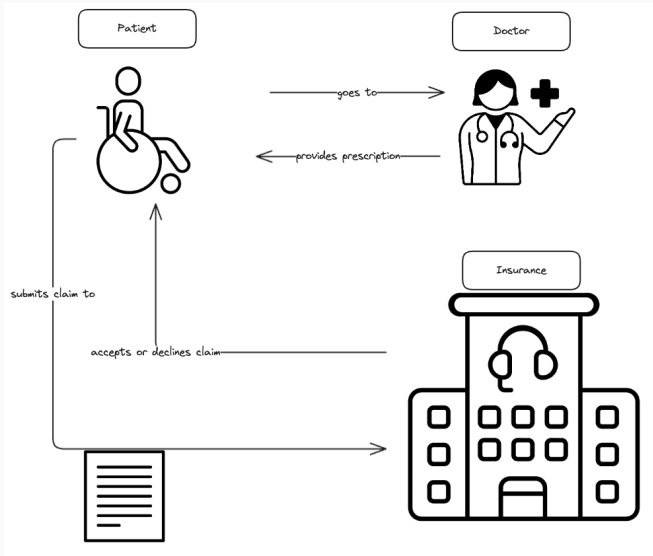
- This is also because we deal with large amounts of data.
- If you do not use the right tools, you will spend most of your time **waiting** on data.

Why so much time on data?

Two case studies

- Automating a business process for an insurance company
- Training an LLM

Remember the case study from last week?



The data quickly gets complicated

claim_id	timestamp	age	obj_id	prescription	tfp	price	decision
1	09:27:43 2024-05-05	80	1	yes	2	100	
1	13:30:34 2024-05-05	80	1	yes	3	100	
1	16:25:30 2024-05-06	80	1	yes	3	100	1

and because of many different stages, the amount of data is huge

claim_id	timestamp	age	obj_id	prescription	tfp	price	decision
2	10:22:22 2024-05-05	25	1			4 100	
2	10:22:22 2024-05-05	25	2	yes		4 5000	
2	13:12:12 2024-05-06	25	1	yes		4 100	
2	13:12:12 2024-05-06	25	2	yes		4 5000	
2	15:09:10 2024-05-07	25	1	yes		4 50	1
2	15:09:10 2024-05-07	25	2	yes		4 5000	2

The data we worked with

We had to understand:

- > 20 tables in a (relational) data base
- > 500 mil. rows
- > 300 columns

Data is also important for AI

Today's SOTA models depend mainly on data (and compute + parameters). E.g. Llama 3.1 was trained with:

- pre-training: 15 trillion tokens of data
- fine-tuning: publicly available instruction datasets, as well as over 25M synthetically generated examples

→ **Being able to handle this amount of data is key!**

How to speed things up

We will be loading >178 million rows in <2 second

Starting with Polars

We will be loading >178 million rows in <2 second

Starting with Polars

```
# Let's load NYC taxi dataset with Polars
nyc = pl.scan_parquet("~/Desktop/nyc-taxi/**/*.parquet", hive_partitioning=True)
q1 = (
    nyc.group_by(["passenger_count"])
    .agg([pl.mean("tip_amount").alias("mean_tip")])
    .sort("passenger_count")
)

tic = time.time()
dat1 = q1.collect()
toc = time.time()
print(f"Time: {toc - tic}")
# Output: Time: 1.4287219047546387
```

- Not quite under one second, but Polars is powerful!
- NB: We are already using a few tricks here with Polars that are not available with Pandas.

Hugging Face suggest Polars for datasets

The screenshot shows the Hugging Face interface for the 'wikimedia/wikipedia' dataset. The page includes a search bar, navigation tabs (Models, Datasets, Spaces, Posts), and a banner for joining an organization. The dataset page displays various filters (Tasks, Languages, Formats, Sub-tasks, Libraries, License) and a 'Dataset card' tab. The 'Dataset Viewer' section shows the dataset's subset (20231101.ab) and a table of data. A yellow arrow points from the text 'suggesting Polars' to the 'Polars' library in the 'Libraries' dropdown menu. The dropdown menu also lists 'Datasets', 'Dask', and 'Croissant'.

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts

Hugging Face is way more fun with friends and colleagues! Join an organization

Datasets: wikimedia/wikipedia like 563 Follow Wikimedia 25

Tasks: Text Generation Fill-Mask Modalities: Text Formats: parquet Sub-tasks: language-modeling masked-language-modeling

Languages: Abkhaz Achinese Adyghe + 319 Size: 10M - 100M Libraries: Datasets Dask Croissant + 1 License: cc-by-sa-3.0

Dataset card Viewer Files and versions Community 66

Dataset Viewer Auto-converted to Parquet API Embed Full Screen Viewer

Subset (223) 20231101.ab · 6.15k rows Split (1) train · 6.15k rows

Search this dataset SQL Console

id	url	title	text
807	https://ab.wikipedia.org/wiki/1883D08B828D08B1D1888D087D1888D083999D08B80	Ацсуа бусааа	Ацсуа бусааа, ацсуа хамтгаар
	https://ab.wikipedia.org/wiki/1883D08B828D08B1D1888D087D1888D083999D08B80	Ацсуа бусааа	Ацсуа бусааа, ацсуа хамтгаар

Downloads last month Use this dataset

Libraries

- Datasets
- Dask
- Croissant
- Polars**

71.8 GB

Loading >178 million rows in <1 second with a database mngmt. framework

Let's use a lightweight database management framework:

Loading >178 million rows in <1 second with a database mngmt. framework

Let's use a lightweight database management framework:

```
import duckdb as ddb
con = ddb.connect("nyc_taxi.ddb")
query = """SELECT
    t0.passenger_count,
    AVG(t0.tip_amount) AS mean_tip
FROM main.nyc_taxi AS t0
GROUP BY
    1
"""

tic = time.time()
con.sql(query).show()
toc = time.time()
print(f"Time: {toc - tic}")
# Output: Time: 0.2068800926
```

Loading >178 million rows in <1 second with a database mngmt. framework

Let's use a lightweight database management framework:

```
import duckdb as ddb
con = ddb.connect("nyc_taxi.ddb")
query = """SELECT
    t0.passenger_count,
    AVG(t0.tip_amount) AS mean_tip
FROM main.nyc_taxi AS t0
GROUP BY
    1
    """

tic = time.time()
con.sql(query).show()
toc = time.time()
print(f"Time: {toc - tic}")
# Output: Time: 0.2068800926
```

- Okay, it's faster, but isn't Polars enough?
 - Another framework and a new programming language for 1 second?
- We will go through the things we can do to speed things up with Polars and Pandas to get an intuition for what database management systems do for you
- And then also talk about other reasons for having your data in a database

Why you should not work with .csv



Figure 2: [FX Markets](#) and [FT](#) reports.

Only use what you need

Choose your columns already when loading the data

So far, we just loaded in all the data:

```
1 df = pd.read_csv(DATA_PATH)
```

```
2
```

Choose your columns already when loading the data

So far, we just loaded in all the data:

```
1 df = pd.read_csv(DATA_PATH)
```

2

Instead, we can deliberately choose the columns we will work with:

```
1 df = pd.read_csv(DATA_PATH, usecols=['column1', 'column2',  
2 'column5'])
```

When exploring, only load a subset of rows

Use the `nrows` argument to limit how much data you are loading in:

```
1 df = pd.read_csv(DATA_PATH, nrows=10)
```

```
2
```

Choose the right data types

Python's data types

Python has the following built-in data types:

- Text type: `str`
- Numeric types: `int`, `float`, `complex`
- Sequence types: `list`, `tuple`, `range`
- Mapping type: `dict`
- Set types: `set`, `frozenset`
- Boolean type: `bool`
- Binary types: `bytes`, `bytearray`, `memoryview`
- None type: `NoneType`

See [W3 Schools - Data Types](#)

Most important examples

- Text type: `str`
 - Example: `x = "Hello World"`
- Numeric types: `int`, `float`, `complex`
 - `int` example: `x = 20`
 - `float` example: `x = 20.5`
- Sequence types: `list`, `tuple`, `range`
 - `list` example: `x = ["apple", "banana", "cherry"]`
 - `tuple` example: `x = ("apple", "banana", "cherry")`
 - `range` example: `x = range(6)`
- Mapping type: `dict`
 - Example: `x = {"name" : "John", "age" : 36}`
- Set types: `set`
 - `set` example: `x = {"apple", "banana", "cherry"}`
- Boolean type: `bool`
 - Example: `x = True`
- None type: `NoneType`
 - Example: `x = None`

Why do we care about data types?

It might seem dull, but there are good reasons we care about data types:

- Data types tell Python how to work on the data. They define what operations can be performed.
 - We can do: `'Hello class!' * 2` $\rightarrow$ `'Hello class!Hello class!'`

Why do we care about data types?

It might seem dull, but there are good reasons we care about data types:

- Data types tell Python how to work on the data. They define what operations can be performed.
 - We can do: `'Hello class!' * 2` → `'Hello class!Hello class!'`
 - But we can't do: `'Hello class' + 2` → `TypeError: can only concatenate str (not "int") to str`

Why do we care about data types?

It might seem dull, but there are good reasons we care about data types:

- Data types tell Python how to work on the data. They define what operations can be performed.
 - We can do: `'Hello class!' * 2` → `'Hello class!Hello class!'`
 - But we can't do: `'Hello class' + 2` → `TypeError: can only concatenate str (not "int") to str`
 - But we can do: `'Hello class!' + str(2)` → `'Hello class!2'`
 - Knowing about data types will help you debug your programs
 - With all data types, you can tell Python explicitly what it should infer like we did above with `str(2)`

Why do we care about data types today?

Today, we also care because:

- **Storing data:** storing columns as certain data types can reduce memory requirements

An example: int16 vs. int32

```
1 dtype_dict = {'column1': 'int16',  
2               'column2': 'int32'}  
3  
4 df = pd.read_csv('file.csv', dtype=dtype_dict)  
5
```

1. Memory Size:

- `np.int16`: Uses 16 bits (2 bytes) to store an integer.
- `np.int32`: Uses 32 bits (4 bytes) to store an integer.

2. Range of Values:

- `np.int16`: integers from -32,768 to 32,767.
- `np.int32`: integers from -2,147,483,648 to 2,147,483,647.

3. Use Cases:

- Use `np.int16` when you need to save memory and your integers fall within the smaller range.

Optimise your queries

The order in which you wrangle matters

Note: Examples from calmcode.io

```
1      pd.read_csv('path')
2      .groupby('city')
3      .agg(n_adresses=('adress', 'nunique'),
4           n_people=('name', 'size'))
5      .loc[lambda d: d['city'] == 'Amsterdam'])
6
```

The order in which you wrangle matters

Note: Examples from calmcode.io

```
1 pd.read_csv('path')
2   .groupby('city')
3   .agg(n_adresses=('adress', 'nunique'),
4        n_people=('name', 'size'))
5   .loc[lambda d: d['city'] == 'Amsterdam'])
6
```

Instead

```
1 pd.read_csv('path', usecols=['city', 'adress', 'name'])
2   .loc[lambda d: d['city'] == 'Amsterdam'])
3   .groupby('city')
4   .agg(n_adresses=('adress', 'nunique'),
5        n_people=('name', 'size'))
6
```

Leave the order to the library

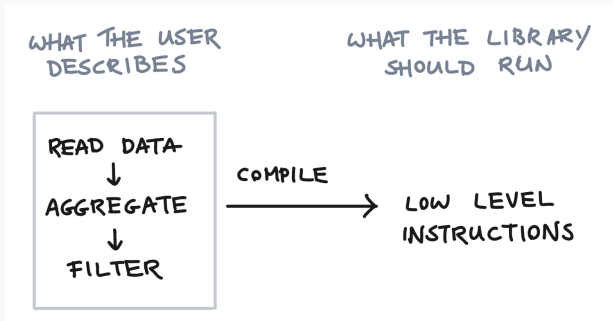


Figure 3: From Polars tutorial by calmcode.io

Polars does this with lazy evaluation

Eager Evaluation:

- Operations are executed immediately
- Results are computed and stored in memory right away

```
1 pl.read_csv("example.csv")
```

```
2
```

Polars does this with lazy evaluation

Eager Evaluation:

- Operations are executed immediately
- Results are computed and stored in memory right away

```
1 pl.read_csv("example.csv")
```

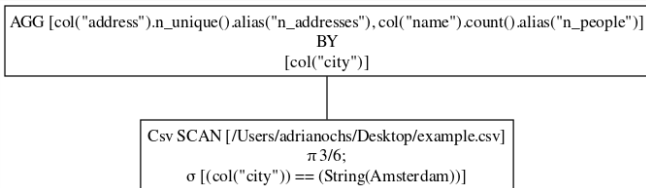
Lazy Evaluation:

- Operations are deferred until explicitly executed
- Query plan is built first, then optimised before execution
- Better for large datasets and complex operations

```
1 lazy_frame = pl.scan_csv("example.csv").select("col_1") #  
  create query  
2 eager_frame = lazy_frame.collect() # execute the optimized  
  query
```

See how polars optimises the query from before

```
1 query = (pl.scan_csv("~/Desktop/example.csv")
2   .group_by("city")
3   .agg(n_addresses=pl.col("address").n_unique(),
4       n_people=pl.col("name").count())
5   .filter(pl.col("city") == "Amsterdam"))
6 query.show_graph()
```



- Optimised plan should be read from bottom to top
- σ stands for filtering and indicates any filter conditions
- π for projection and indicates choosing a subset of columns

A different illustration of the query optimisation

```
1 print(query.explain())
```

```
2
```

```
AGGREGATE
  [col("address").n_unique().alias("n_addresses"), col("name").count().alias("n_people")] BY [col("city")] FROM
Csv SCAN [/Users/adrianochs/Desktop/example.csv]
PROJECT 3/6 COLUMNS
SELECTION: [(col("city")) == (String(Amsterdam))]
```

- Query optimiser applies filter while the CSV is read from **disk** rather than reading the entire file into memory and then applying the filter.
 - This optimization is called Predicate Pushdown
 - "Pushing down" filtering conditions (predicates) as close as possible to the data source before processing the rest of the query

A different illustration of the query optimisation

```
1 print(query.explain())
```

```
2
```

```
AGGREGATE
[col("address").n_unique().alias("n_addresses"), col("name").count().alias("n_people")] BY [col("city")] FROM
Csv SCAN [/Users/adrianochs/Desktop/example.csv]
PROJECT 3/6 COLUMNS
SELECTION: [(col("city") == (String(Amsterdam)))]
```

- Query optimiser applies filter while the CSV is read from **disk** rather than reading the entire file into memory and then applying the filter.
 - This optimization is called Predicate Pushdown
 - "Pushing down" filtering conditions (predicates) as close as possible to the data source before processing the rest of the query
- Similarly, Polars selects which columns should be read in rather than loading the whole data set into memory
 - This optimisation is called Projection Pushdown
 - c.f. linear algebra projection matrix projecting onto lower dimensional space

In-memory vs. on disk

What if the data is still out of memory

We can do lazy evaluation, but at some point we load the result into memory:

```
1 lazy_frame = pl.scan_csv("example.csv").select("col_1")  
2 eager_frame = lazy_frame.collect()  
3
```

What if the data is still out of memory

We can do lazy evaluation, but at some point we load the result into memory:

```
1 lazy_frame = pl.scan_csv("example.csv").select("col_1")  
2 eager_frame = lazy_frame.collect()  
3
```

The problem is: we need to load the data into memory (RAM) to process it.

Loading & processing your data in chunks

When your data is larger than your RAM ("out of memory"), you can load and operate on it in chunks:

```
1  # Initialize list to store processed chunks
2  processed_chunks = []
3
4  # Read and process data in chunks
5  chunk_size = 1000
6  for chunk in pd.read_csv('big_data.csv', chunksize=
7                           chunk_size):
8      # Process chunk by dividing values by 2
9      chunk['value'] = chunk['value'] / 2
10
11     # Store processed chunk
12     processed_chunks.append(chunk)
13
14 # Combine all processed chunks into final result
15 final_result = pd.concat(processed_chunks, ignore_index=True
                             )
```

The same in Polars

```
1  # Read and process data using Polars streaming
2  result = (
3      pl.scan_csv('big_data.csv')
4      .with_columns(pl.col('value') / 2)
5      .collect(streaming=True)) # Use streaming in collect to
6  process in chunks
```

The same in Polars

```
1  # Read and process data using Polars streaming
2  result = (
3      pl.scan_csv('big_data.csv')
4      .with_columns(pl.col('value') / 2)
5      .collect(streaming=True)) # Use streaming in collect to
6  process in chunks
```

Size of the batches is determined by:

- the number of CPUs on your machine
- the amount of memory each row in your query requires

What if the result does not fit into memory either?

We write the output directly to a file on disk using the sink_ methods:

```
1 query.sink_parquet("big_data.parquet")
```

```
2
```

- This is actually very useful, and something that only database frameworks but not data wrangling frameworks could do.

(Relational) Database management systems & SQL

Database management systems to optimise the data load & processing

- A big part of a database management system is how chunking, predicate or projection Pushdowns, and other data load optimisations are handled

Database management systems to optimise the data load & processing

- A big part of a database management system is how chunking, predicate or projection Pushdowns, and other data load optimisations are handled
- Databases contain more information on your data (relationships among tables, indexes, etc.) to speed up the data retrieval and make storage more secure

Database management systems to optimise the data load & processing

- A big part of a database management system is how chunking, predicate or projection Pushdowns, and other data load optimisations are handled
- Databases contain more information on your data (relationships among tables, indexes, etc.) to speed up the data retrieval and make storage more secure
- And you talk to that database using a special programming language called SQL (structured query language)

Database management systems to optimise the data load & processing

- A big part of a database management system is how chunking, predicate or projection Pushdowns, and other data load optimisations are handled
- Databases contain more information on your data (relationships among tables, indexes, etc.) to speed up the data retrieval and make storage more secure
- And you talk to that database using a special programming language called SQL (structured query language)

```
1      SELECT name, age FROM users WHERE city = 'Seattle'  
2
```

Like before, this does

- Predicate Pushdown (WHERE statement is a filter)
- Projection Pushdown (SELECT chooses the columns that should be loaded)

Databases also organise and store your data securely

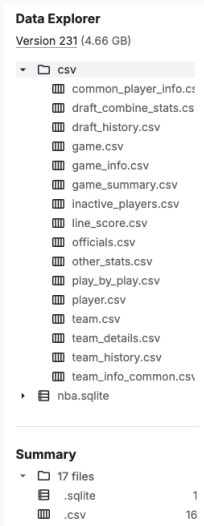


Figure 4: Kaggle NBA data

ER diagram of a data base

A relational database is a type of database that stores and organises data in tables with predefined relationships between them.

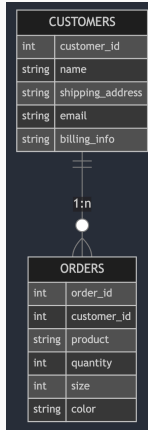
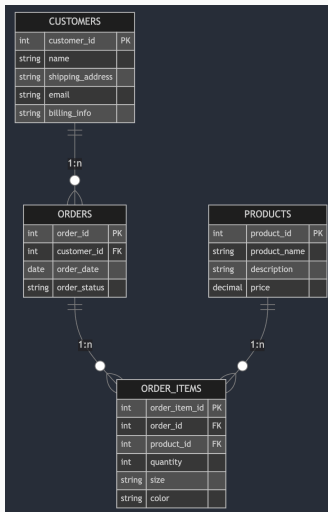


Figure 5: Entity relationship (ER) diagram of database

Your Amazon shopping in a relational database



Introducing SQL

- SQL = Structured Query Language
- SQL is a programming language to store and work with data in a relation database.
- Optimised for data tasks
 - the query processes creates a plan for retrieving or writing data in the most effective way
 - answer questions like "how many users visited our website today" blazingly fast
- Good thing: it's fairly easy!

How a query looks

```
SELECT *  
From cats
```

Run query

Query results

owner	name
1	daisy
1	dragonsnap
3	buttercup
4	rose

Figure 6: From J. Evans "Become a Select Star"

How SELECT works

A SQL database contains a bunch of tables.

sales

client	item
~	~
~	~
~	~
~	~

clients

id	name
~	~
~	~
~	~
~	~

cats

owner	name
~	~
~	~
~	~
~	~

Every SELECT query takes data from those tables and outputs a table of results.

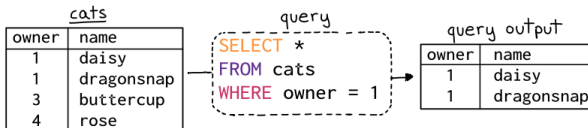


Figure 7: From J. Evans "Become a Select Star"

How a slightly more complicated query looks

```
SELECT *  
From cats  
Where owner != 1
```

Run query

Query results

owner	name
3	buttercup
4	rose

Figure 8: From J. Evans "Become a Select Star"

How a more complicated query looks

```
SELECT owner, count(*)  
FROM cats  
WHERE owner != 3  
GROUP BY owner  
HAVING COUNT(*) = 2  
ORDER BY owner DESC
```

Figure 9: From J. Evans "Become a Select Star"

```
SELECT owner, count(*)  
From cats  
WHERE owner != 3  
GROUP BY owner  
HAVING count(*) = 2  
ORDER BY owner DESC
```

Run query

Query results

owner	count(*)
-------	----------

1	2
---	---

Figure 10: From J. Evans "Become a Select Star"

Teaching SQL syntax in one slide

- Limit your queries (i.e. only show first N rows) to test your code.
 - `SELECT * FROM my_table LIMIT 1`
- SQL is not case sensitive, but it is best practice to write the commands in uppercase.
- Queries have to be written in a specific order:
 - `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`
 - WHERE filters individual rows before they're grouped
 - HAVING filters groups after the GROUP BY
- Equality is written with one "=" and not "col_1 == 2"
- To check whether a column has NULL entries, write "col_1 IS NULL" rather than "col_1 = NULL"

Try it out

Go to: <https://sql-playground.wizardzines.com/>

Answer the following questions:

1. How many rows does the "people" table have?
2. What are the column names of the sales table?
3. How many "diaper" entries does the column "event" have in the "baby log" table?

Solution 1: nr. rows in people table

```
SELECT count(*)  
FROM people
```

Run query

Query results

count(*)

4

Solution 2: col names of sales table

```
SELECT *  
FROM sales  
LIMIT 1
```

Run query

Query results

month	item	client	price	day
Jan	catnip	1	5	2

NB: this is PostgreSQL dialect. T-SQL dialect has "SELECT TOP 1 *"
Instead of "SELECT ... LIMIT"

Solution 3: nr. of "diaper" entries in baby_log table

```
SELECT count(*)  
FROM baby_log  
WHERE event = 'diaper'
```

Run query

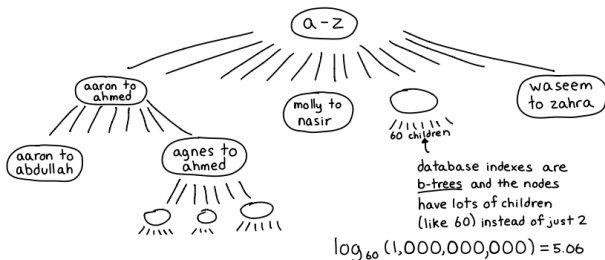
Query results

count(*)
3

DBs use indexes to make your data retrieval quicker

```
SELECT * FROM cats WHERE name = 'mr darcy'
```

Indexes are a tree structure that makes it faster to find rows. Here's what an index on the 'name' column might look like.

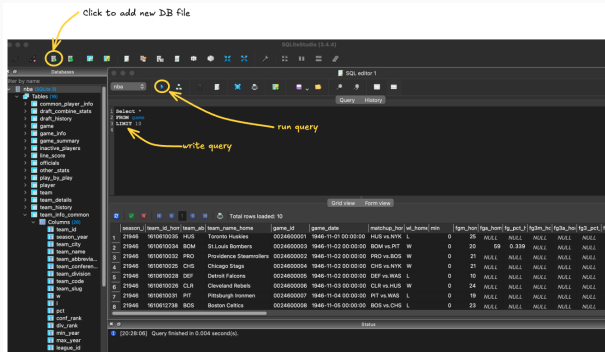


This means that if you have 1 billion names to look through, you'll only need to look at maybe ⑤ nodes in the index to find the name you're looking for (5 is a lot less than 1 billion!!!).

Figure 11: From J. Evans "Become a Select Star"

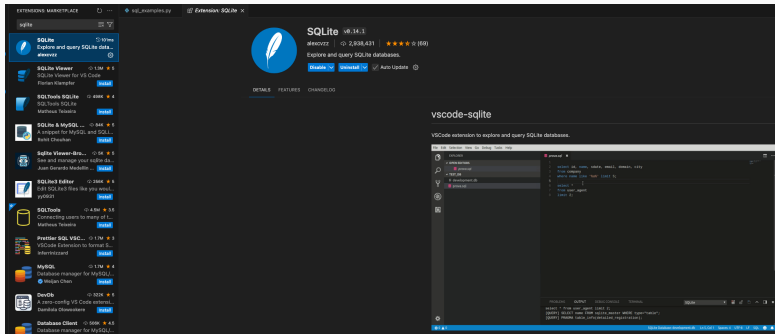
Python and DBs

How to connect to a DB - using an IDE

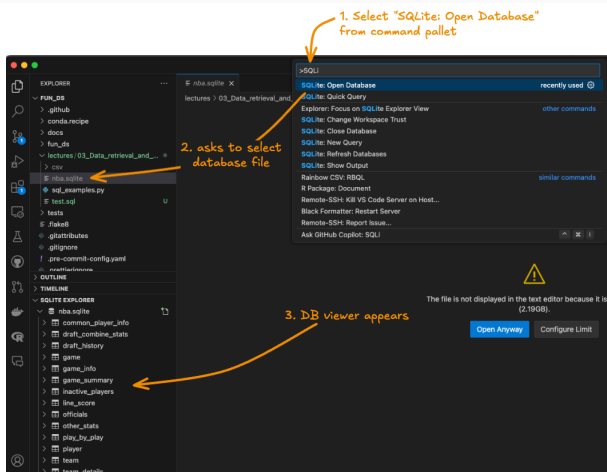


- This is [SQLiteStudio](#)
- There are many others: [DBeaver](#), [MySQL](#), [SSMS](#)

How to connect to a DB - VS Code extension

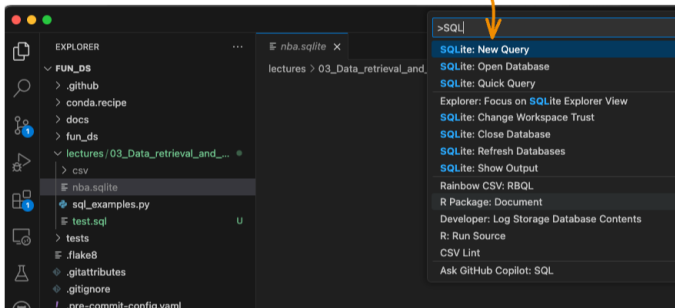


Using SQLite extension to see the DB

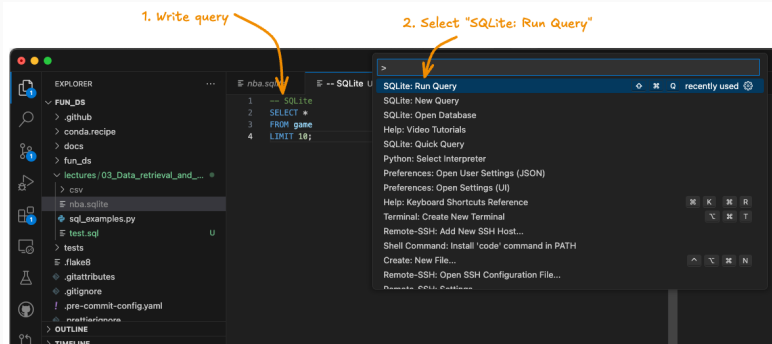


Using SQLite extension to write & run SQL queries

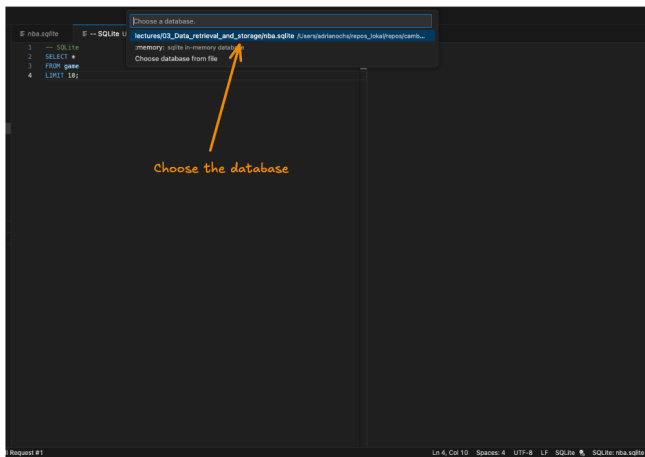
Select "SQLite: New Query" to create a .sql file



Using SQLite extension to write & run SQL queries



Using SQLite extension to write & run SQL queries



Using SQLite extension to view your results

The screenshot shows a code editor with a file explorer on the left and a query editor on the right. The file explorer shows a project structure with files like `fun_ds`, `.github`, `conda.recipe`, `docs`, `fun_ds`, `lectures/03_Data_retrieval_and_...`, `csv`, `nba.sqlite`, `sql_examples.py`, `test.sql`, `tests`, `flake8`, `gitattributes`, `gitignore`, `pre-commit-config.yaml`, `pytest.ini`, `OUTLINE`, `TIMELINE`, `SQLITE EXPLORER`, `nba.sqlite`, and `common_player_info`.

The query editor shows the following SQL query:

```
1 -- SQLite
2 SELECT *
3 FROM game
4 LIMIT 10;
```

The results of the query are displayed in a table with the following columns: `season_id`, `team_id_home`, `team_abbreviation_home`, `team_name_home`, `game_id`, and `game_id`. The table contains 10 rows of data, showing game results for the 2014-2015 season.

season_id	team_id_home	team_abbreviation_home	team_name_home	game_id	game_id
21946	1610610035	MUS	Toronto Huskies	0024600001	1946
21946	1610610034	BOM	St. Louis Bombers	0024600003	1946
21946	1610610032	PRD	Providence Steamrollers	0024600002	1946
21946	1610610025	CHS	Chicago Stags	0024600004	1946
21946	1610610028	DEF	Detroit Falcons	0024600005	1946
21946	1610610026	CLR	Cleveland Rebels	0024600006	1946
21946	1610610031	PIT	Pittsburgh Ironmen	0024600007	1946
21946	1610612738	BOS	Boston Celtics	0024600008	1946
21946	1610610028	DEF	Detroit Falcons	0024600009	1946
21946	1610610032	PRD	Providence Steamrollers	0024600011	1946

How to connect to a DB with Python

```
1 import sqlite3 as sql
2 con = sql.connect("nba.sqlite")
3 df_games = pd.read_sql("SELECT * FROM game LIMIT 10",
4                          con)
5
```

- We execute the SQL query with Pandas loading the end result into memory

Intermediate results on disk

```
# what where the top 10 games with the most plays?
query = """
CREATE TEMPORARY TABLE plays_per_game AS
  SELECT game_id, COUNT(*) as number_of_plays
  FROM play_by_play
  GROUP BY game_id
"""

con.execute(query)

query = """
SELECT *
FROM game as g
LEFT JOIN plays_per_game as ppg
ON g.game_id = ppg.game_id
WHERE season_type = 'Playoffs'
ORDER BY ppg.number_of_plays DESC
LIMIT 10
"""

pd.read_sql(query, con)
```

- Create a temporary result on disk (not with Pandas)
- Use it for the final calculation

Intermediate results on disk

```
# what where the top 10 games with the most plays?
query = """
CREATE TEMPORARY TABLE plays_per_game AS
  SELECT game_id, COUNT(*) as number_of_plays
  FROM play_by_play
  GROUP BY game_id
"""

con.execute(query)

query = """
SELECT *
FROM game as g
LEFT JOIN plays_per_game as ppg
ON g.game_id = ppg.game_id
WHERE season_type = 'Playoffs'
ORDER BY ppg.number_of_plays DESC
LIMIT 10
"""

pd.read_sql(query, con)
```

- Create a temporary result on disk (not with Pandas)
- Use it for the final calculation
- Temporary results are only stored during your session
- You can also create a "View" which is stored even when you close the session

You can write Pandas data frames to the DB

```
1 import sqlite3 as sql
2 con = sql.connect("nba.sqlite")
3 df_games.to_sql("top_10", con)
4
```

or create a new one with sqlite3

```
1 con = sql.connect("nba.sqlite")
2 con.execute("CREATE TABLE salaries(player_id, year,
3 salary)")
```

Or easily build a new database

```
1 con_new = sql.connect("new.sqlite")  
2 con_new.execute("CREATE TABLE movie(title, score)")  
3
```

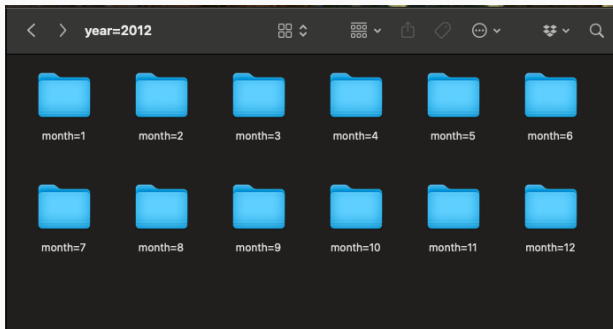
Close your connection

When you are done, close your connection:

```
1     con_new.close()
```

```
2
```

DuckDB as an alternative



- These folders contain one parquet file with NYC taxi data for each month
- DuckDB can query parquet and csv using SQL
- DuckDB can create a database from these files

DuckDB can query parquet files directly

```
import duckdb as ddb

query_2 = """SELECT
    t0.passenger_count,
    AVG(t0.tip_amount) AS mean_tip
FROM read_parquet('nyc-taxi/**/*.*parquet') AS t0
GROUP BY
    1
    """

con = ddb.connect()
con.sql(query_2).show()
```

DuckDB can build a DB from parquet

```
# Turn the parquet files into a DuckDB table
con = ddb.connect("nyc_taxi.ddb")
# Write the parquet files to a table
con.sql("CREATE TABLE nyc_taxi AS SELECT * FROM read_parquet('nyc-taxi/**/*.parquet')")
```

- First, create a new database called nyc-taxi
- Then create the new table by reading in the data

This works seamlessly because of hive partitioning

```
orders
├── year=2021
│   ├── month=1
│   │   ├── file1.parquet
│   │   └── file2.parquet
│   └── month=2
│       └── file3.parquet
└── year=2022
    ├── month=11
    │   ├── file4.parquet
    │   └── file5.parquet
    └── month=12
        └── file6.parquet
```

- Hive partitioning is a partitioning strategy that is used to split a table into multiple files based on partition keys ([DuckDB hive partitioning](#))

Another way: SQLAlchemy allows for more "Pythonic" code

```
>>> from sqlalchemy import select
>>> stmt = select(user_table).where(user_table.c.name == "spongebob")
>>> print(stmt)

SELECT user_account.id, user_account.name, user_account.fullname
FROM user_account
WHERE user_account.name = :name_1
```

Figure 12: [SQLAlchemy](#)

Different DBMS have different SQL dialects:

- Microsoft SQL Server: T-SQL (Transact-SQL)
 - Uses "SELECT TOP 10 *" rather than "SELECT * FROM table LIMIT 10"
- PostgreSQL: PL/pgSQL (Procedural Language/PostgreSQL)
- MySQL: SQL with MySQL extensions
- Oracle: PL/SQL (Procedural Language/SQL)

Different DBMS have different SQL dialects:

- Microsoft SQL Server: T-SQL (Transact-SQL)
 - Uses "SELECT TOP 10 *" rather than "SELECT * FROM table LIMIT 10"
- PostgreSQL: PL/pgSQL (Procedural Language/PostgreSQL)
- MySQL: SQL with MySQL extensions
- Oracle: PL/SQL (Procedural Language/SQL)

→ With SQL Alchemy you stay flexible.

Case study: different dialects

- You might have a big database to retrieve your training data from
- For inference (i.e. during prediction), however, your model has to ingest CSV files or work with a lightweight database

Case study: different dialects

- You might have a big database to retrieve your training data from
- For inference (i.e. during prediction), however, your model has to ingest CSV files or work with a lightweight database
- This means that your data cleaning and wrangling might have to be rewritten if you
 - have to use a different SQL dialect
 - if you use SQL for training data preparation but Pandas and CSV file during prediction

Case study: different dialects

- You might have a big database to retrieve your training data from
- For inference (i.e. during prediction), however, your model has to ingest CSV files or work with a lightweight database
- This means that your data cleaning and wrangling might have to be rewritten if you
 - have to use a different SQL dialect
 - if you use SQL for training data preparation but Pandas and CSV file during prediction
- Think about it upfront to align your code and not have to do the same work twice!

Data storage & Mngt. landscape

So which framework should you choose?

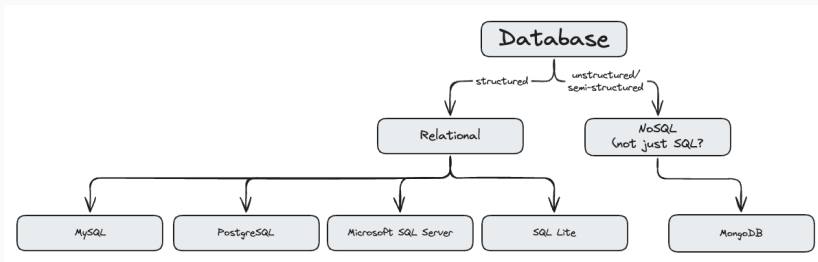


Work with what your company or your client uses.

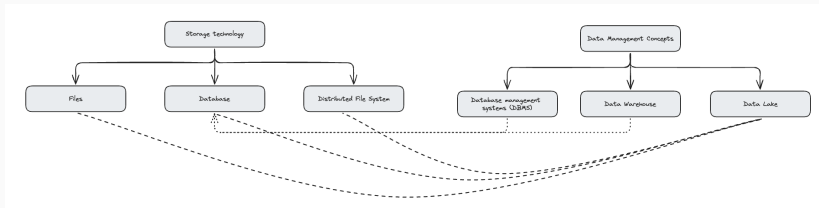
Spoiled with choice



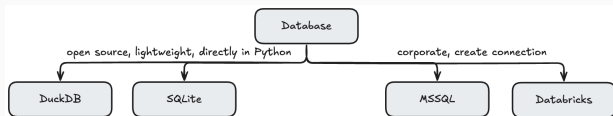
Order according to structured vs. unstructured



Categorise according to storage technology



Lightweight vs corporate



There are over one trillion (1e12) SQLite databases in active use



- Think of SQLite as your go-to for application data storage and simple transactions
 - "Since SQLite is used extensively in every smartphone, and there are more than 4.0 billion (4.0e9) smartphones in active use, each holding hundreds of SQLite database files, it seems likely that there are over one trillion (1e12) SQLite databases in active use."([SQLite](#))
- DuckDB excels at data analysis and processing large datasets quickly (good for DS)

Some SQL resources

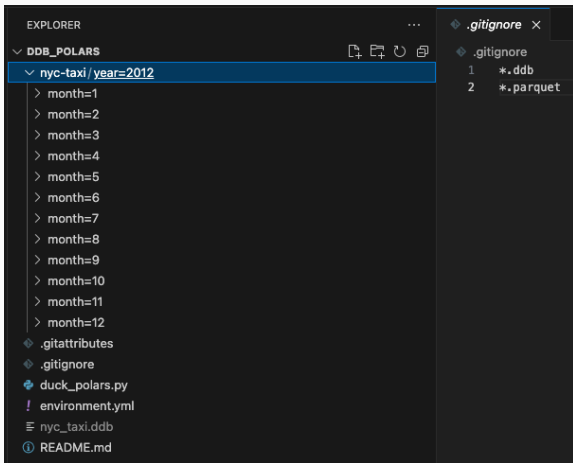
- Learning the syntax is fairly easy and chat bots are of great help
- Some great resources are:
 - [J. Evans "Become a SELECT Star"](#)
 - [W3 schools SQL tutorial](#)
 - [Codeacademy SQL course](#)

Adding to our repo structure:
.gitignore, pre-commit, type hints

.gitignore to ignore files you do not want to push to GitHub

You can tell Git to ignore certain files:

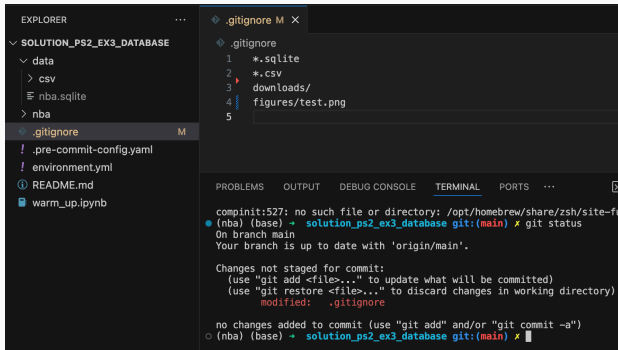
- Large data files
- API keys
- Often your graphs (.png)



.gitignoring file endings, folders, or specific files



Ignored files will not be tracked by Git



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with a `SOLUTION_PS2_EX3_DATABASE` folder containing a `data` subfolder with `csv` and `nba` subfolders. The `nba` folder contains `nba.sqlite` and `.gitignore` files. The `.gitignore` file is open in the editor, showing the following content:

```
.gitignore
1 *.sqlite
2 *.csv
3 downloads/
4 figures/test.png
5
```

The terminal window shows the output of a `git status` command. It indicates that the branch is up to date with 'origin/main' and that there are no changes staged for commit. The output is as follows:

```
compinit:527: no such file or directory: /opt/homebrew/share/zsh/site-fu
• (nba) (base) + solution_ps2_ex3_database git:(main) ✖ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
○ (nba) (base) + solution_ps2_ex3_database git:(main) ✖
```

- I have added the `nba.sqlite` but ignore `.sqlite` files
- Git will not track `.sqlite` files so it does not appear in your staging area
- Avoids you adding and pushing things you do not want to push to GitHub
- Pushing the database would crash because it requires too much space on GitHub

Type hints for better functions

- Good functions specify what should go in and what should come out:

```
1  def multi_two_numbers(a, b):  
2      return a * b  
3
```

Type hints for better functions

- Good functions specify what should go in and what should come out:

```
1 def multi_two_numbers(a, b):  
2     return a * b  
3
```

VS

```
1 def multi_two_numbers(a: int, b: int) -> int:  
2     return a * b  
3
```

Type hints with default arguments

```
1 def greet(name: str = "World") -> str:  
2     return f"Hello, {name}!"  
3
```

Use the typing package for more types

Some types are not available natively, but you can easily extend your typing:

```
1  from typing import List
2  def some_func(a: List[int]) -> List[str]:
3      return [str(x) for x in a]
4
```

- mypy's [type hints cheat sheet](#)

So what happens if:

```
1 def multi_two_numbers(a: int, b: int) -> int:  
2     return a * b  
3
```

```
1 multi_two_numbers(2, "2.0")  
2
```

So what happens if:

```
1 def multi_two_numbers(a: int, b: int) -> int:  
2     return a * b  
3
```

```
1 multi_two_numbers(2, "2.0")  
2
```

- **Nothing** because you can multiply a string with an integer
 - Output: "2.02.0"
- Python's type hints don't enforce types by themselves
- But, they're valuable for documentation
- And you can use type checking libraries to catch issues early

Pre-commit hooks automatically test and prettify your code

- Pre-commit is a Python package
- Pre-commit hooks run before you commit
- They are checking:
 - If your code follows certain standards (line length of 80 or no trailing white space)
 - That you do not install unnecessary libraries
 - Your type hints in functions (if you set any)

How to set up pre-commit hooks

To install precommit, run the following in a virtualenv (and while in a git repo).

```
pip install pre-commit
```

You can confirm everything is installed by running;

```
pre-commit --help
```

To generate a new settings file locally, you need to run;

```
pre-commit sample-config > .pre-commit-config.yaml
```

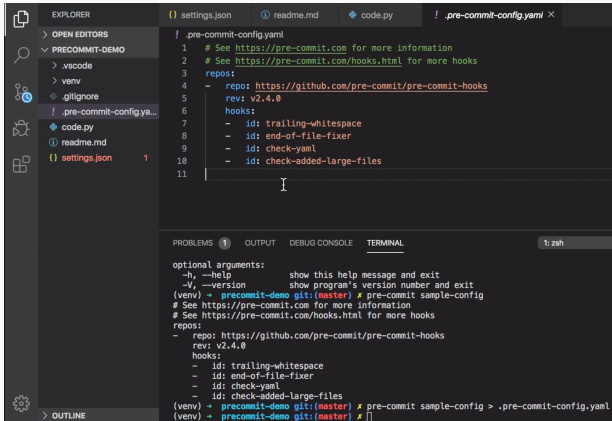
Don't forget to install the hooks.

```
pre-commit install
```

Now you're ready to run the hook.

Figure 13: [calmcode tutorial](#)

Your .pre-commit-config.yaml



The screenshot shows a code editor with a sidebar on the left containing icons for Explorer, Search, Source Control, Run and Debug, and Settings. The Explorer sidebar shows a project named 'PRECOMMIT-DEMO' with files like .vscode, .venv, .gitignore, .pre-commit-config.yaml, code.py, readme.md, and settings.json. The main editor area displays the content of '.pre-commit-config.yaml' with line numbers 1 through 11. The file content is as follows:

```
1 # See https://pre-commit.com for more information
2 # See https://pre-commit.com/hooks.html for more hooks
3 repos:
4 - repo: https://github.com/pre-commit/pre-commit-hooks
5   rev: v2.4.0
6   hooks:
7     - id: trailing-whitespace
8     - id: end-of-file-fixer
9     - id: check-yaml
10    - id: check-added-large-files
11
```

Below the editor, the 'TERMINAL' tab is active, showing the output of a command. The terminal text is:

```
optional arguments:
  -h, --help            show this help message and exit
  -V, --version          show program's version number and exit
(venv) + precommit-demo git:(master) x pre-commit sample-config
# See https://pre-commit.com for more information
# See https://pre-commit.com/hooks.html for more hooks
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v2.4.0
  hooks:
  - id: trailing-whitespace
  - id: end-of-file-fixer
  - id: check-yaml
  - id: check-added-large-files
(venv) + precommit-demo git:(master) x pre-commit sample-config > .pre-commit-config.yaml
(venv) + precommit-demo git:(master) x []
```

Figure 14: calmcode tutorial

Running pre-commit

```
$ pre-commit run --all-files
[INFO] Initializing environment for https://github.com/pre-commit/pre-commit-hooks.
[INFO] Initializing environment for https://github.com/psf/black.
[INFO] Installing environment for https://github.com/pre-commit/pre-commit-hooks.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/psf/black.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
Check Yaml.....Passed
Fix End of Files.....Passed
Trim Trailing Whitespace.....Failed
- hook id: trailing-whitespace
- exit code: 1

Files were modified by this hook. Additional output:

Fixing sample.py

black.....Passed
```

- They will run every time you want to commit
- When they find an error you will have to "git add" them and then run the commit again
- Or if the error is not automatically fixed, you have to first fix it, "git add", and then "git commit" again

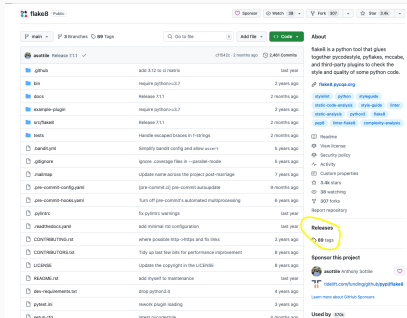
You can install many different hooks by adding to your config

- Look at <https://pre-commit.com/hooks.html> or google "pre-commit hook mypy"
- Black, Flake8 are popular

```
# See https://pre-commit.com for more information
# See https://pre-commit.com/hooks.html for more hooks
repos:
-   repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.4.0
    hooks:
    -   id: trailing-whitespace
    -   id: end-of-file-fixer
    -   id: check-yaml
    -   id: check-added-large-files
    -   id: check-ast
    -   id: check-json
-   repo: https://gitlab.com/pycqa/flake8
    rev: 8f9b4931b9a28896fb43edccb23016a7540f5b82
    hooks:
    -   id: flake8
```

How to add hooks to your .pre-commit-config.yml?

- **repo:** The URL to the repo of the pre-commit hook
- **rev:** That's the release tag of the repo
- **hooks:** The hooks you want to use



Linters vs. code formatters

- Flake8 is a code linter to check your code for:
 - Line length
 - Unused imports
- Black is a code formatter that automatically reformats your code

```
© (fun_ds) (base) * solution_ps2_ex3_database git:(main) x git commit -m "init"
Trim Trailing Whitespace.....Passed
Fix End of Files.....Passed
Check Yaml.....Passed
Check for added large files.....Passed
flake8.....Failed
- hook id: flake8
- exit code: 1

nba/data_loader.py:12:80: E501 line too long (85 > 79 characters)
nba/data_loader.py:34:80: E501 line too long (87 > 79 characters)

mypy.....Passed
nbstripout.....Passed
black.....Failed
- hook id: black
- files were modified by this hook

reformatted nba/data_loader.py

All done! 🎉 🍌 🍌
1 file reformatted, 1 file left unchanged.
```

Clean the cache when you change your config

```
1 pre-commit clean && pre-commit install
```

```
2
```

To update your hooks

Just type in the terminal:

```
1  pre-commit autoupdate
```

```
2
```